

计算机操作系统

Operating Systems

田卫东

March, 2014

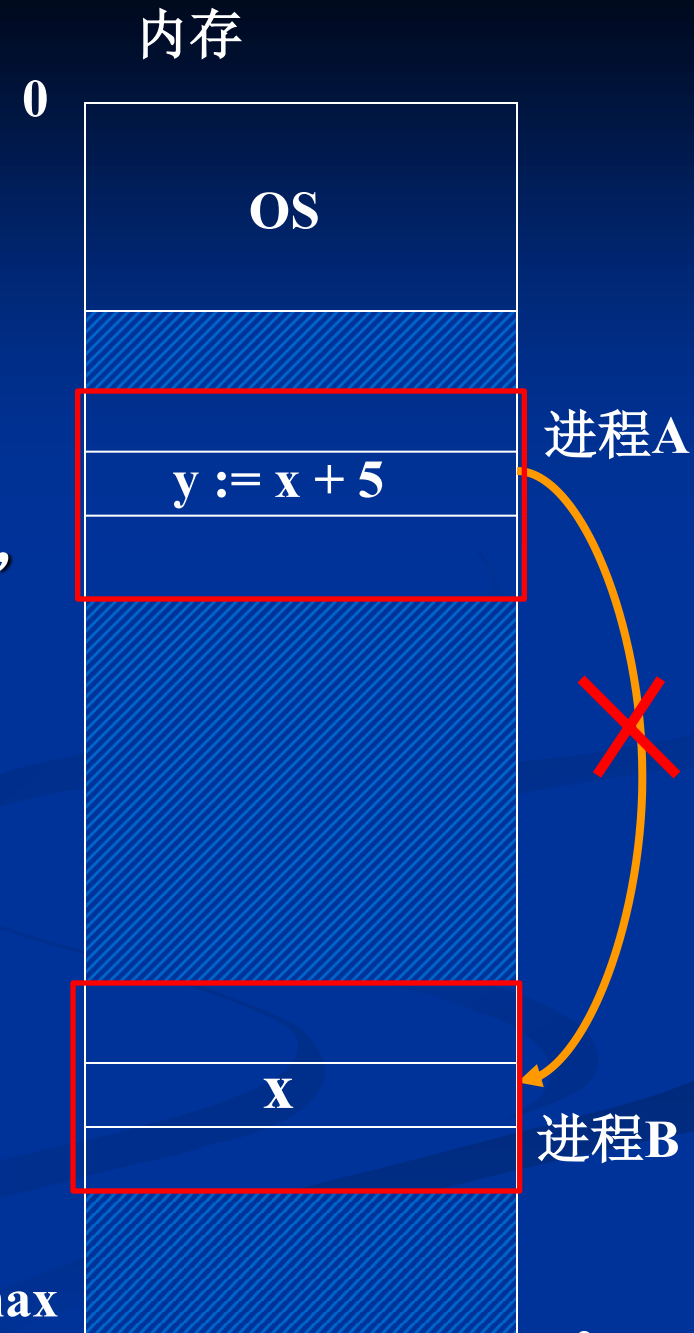
第2章 进程管理

2.5 进程通信

■ 进程间通信的基本概念

Inter Process Communication, IPC

进程通信指进程之间的信息交换，其所交换的信息量，少者是一个状态或数值，多者则是成千上万个字节。



2.5 进程通信

2.5.1 共享存储器系统

(Shared-Memory System)

■ 基本思想

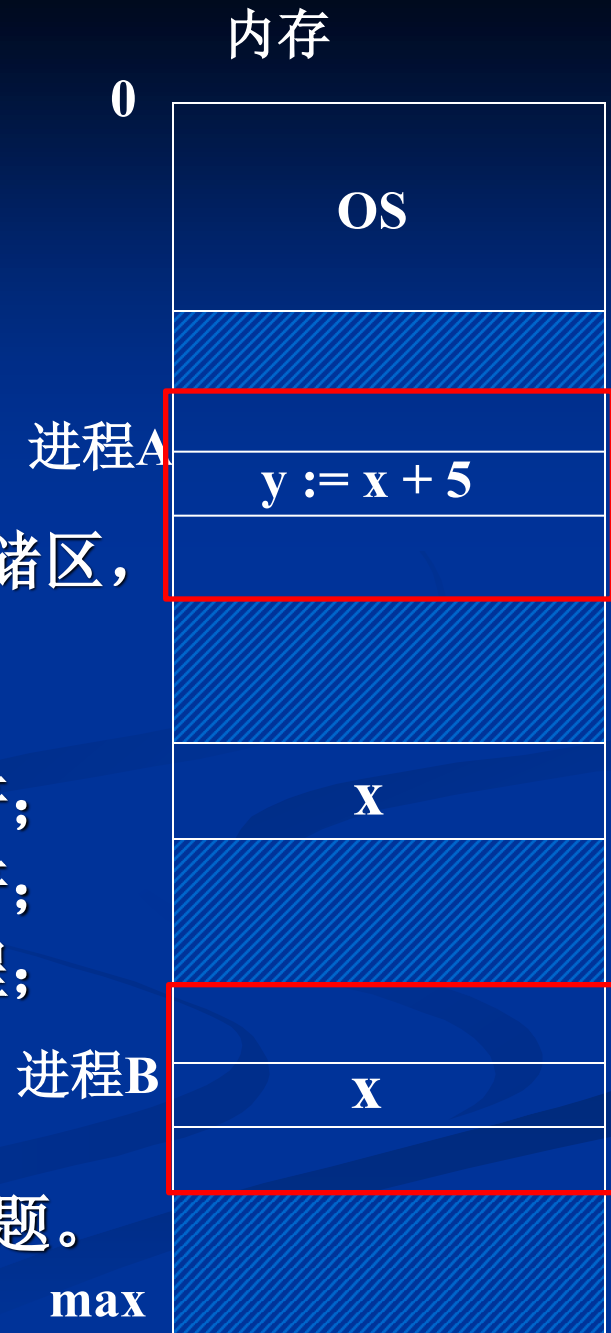
相互通信的进程共享数据结构或共享存储区，进程之间通过这些空间进行通信。

■ 实现过程

- ① 进程申请一片内存，拿到内存描述符；
- ② 如果已分配过，直接获得内存描述符；
- ③ 利用内存描述符将内存连接到本进程；
- ④ 读写

■ 共享存储区的访问同步

共享存储区作为资源，需要考虑共享问题。



2.5 进程通信

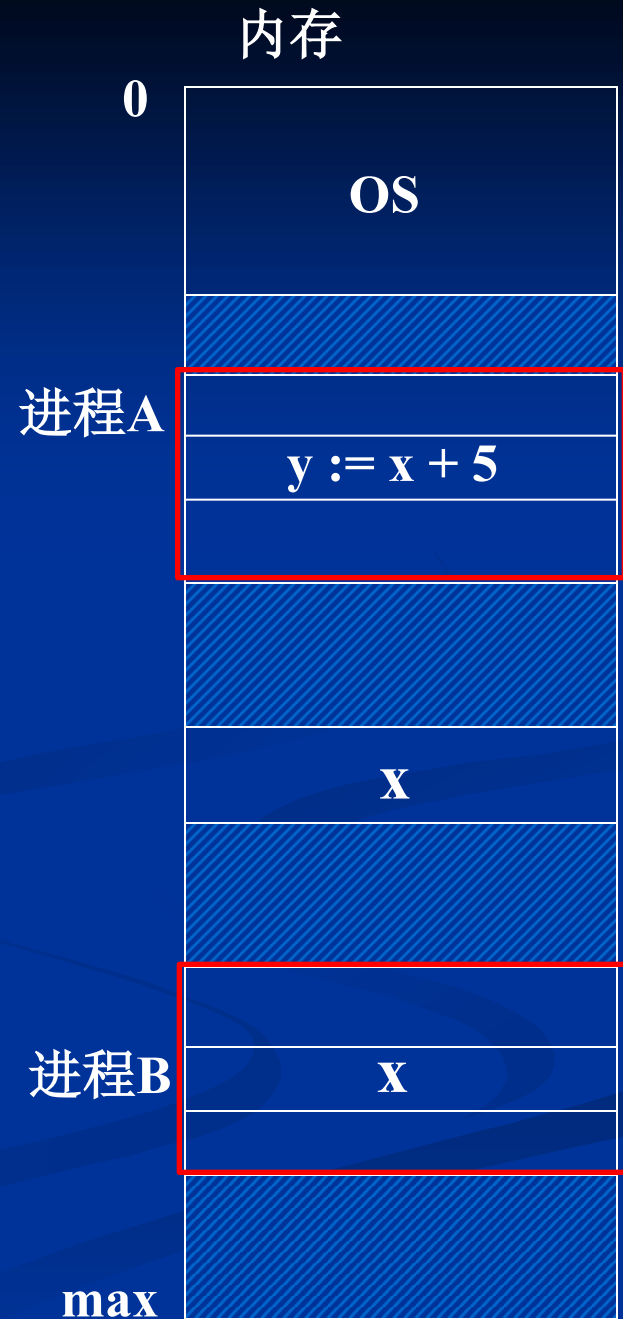
2.5.1 共享存储器系统

■ 分类

- 基于共享数据结构的通信方式
- 基于共享存储区的通信方式

■ 特点

- 适合大量的数据快速交换；
- 程序员负担重



2.5 进程通信

2.5.1 消息通信系统(Message Passing System)

- 基本思想

进程间的数据交换，是以格式化的消息(message)为单位进行通信。

- 消息通信方式的分类

- 直接通信：信息直接传递给接收方

- 在发送时，指定接收方的地址或标识，也可以指定多个接收方或广播式地址

- 在接收时，允许接收来自任意发送方的消息，并在读出消息的同时获取发送方的地址

- 间接通信：借助于收发双方进程之外的共享数据结构作为通信中转，如消息队列。

- 通常收方和发方的数目可以是任意的

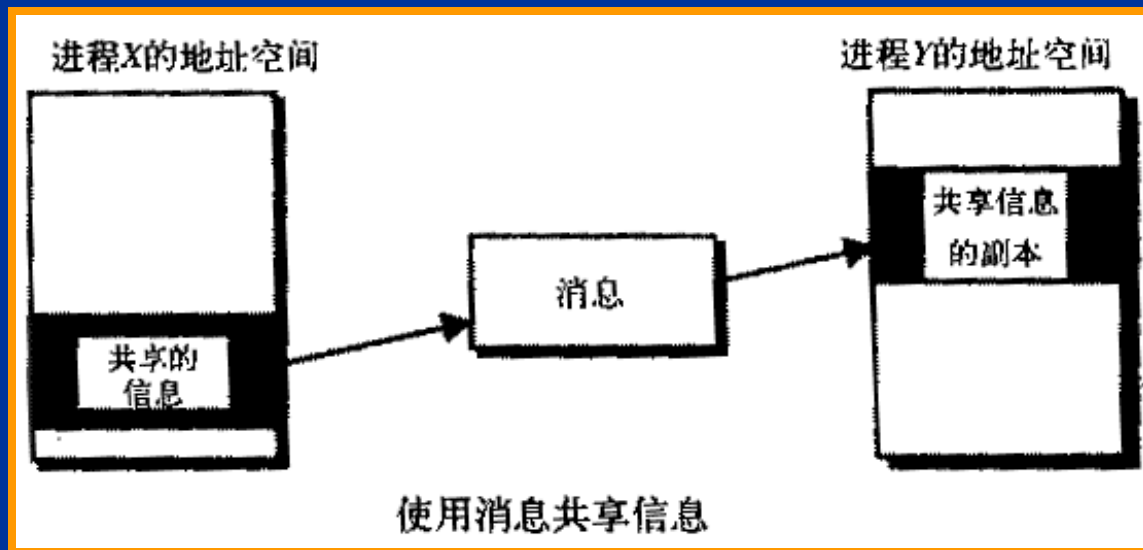
2.5 进程通信

2.5.1 消息通信系统

(1) 直接通信方式

OS提供两个通信原语:

- ① 发送进程: **Send(receiver,msg);** 进程Y
- ② 接收进程: **Receive(sender,msg);**



进程X

内存

0

OS

Receive(X,a);

Send(Y,a);

max

2.5 进程通信

2.5.2 消息通信系统

■ 消息的基本结构

```
type message buffer=record
```

```
    sender ; 发送者进程标识符
```

```
    size   ; 消息长度
```

```
    text   ; 消息正文
```

```
    next   ; 指向下一个消息缓冲区的指针
```

```
end
```


2.5 进程通信

2.5.2 消息通信系统

(2) 间接通信方式:信箱通信

① 发送进程: **Send(mailbox,msg);**

② 接收进程: **Receive(mailbox,msg);**

■ 信箱 (mailbox)

HEAD

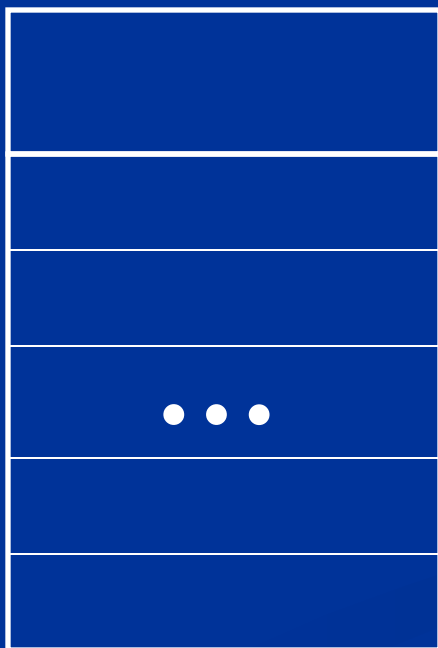
0

1

BODY

...

n



内存

0

OS

进程A

Receive(mail,x);

进程B

Send(mail,x);

max

2.5 进程通信

2.5.2 消息通信系统

■ 信箱分类

- 公有信箱;
- 私有信箱;
- 共享信箱;

■ 特点

- 操作系统隐藏通信细节;
- 通信程序简单;
- 可以跨网络传输;
- 灵活

■ 进程同步

- 发送阻塞、接收阻塞;
- 发送不阻塞、接收阻塞;
- 发送不阻塞、接收不阻塞;



2.5 进程通信

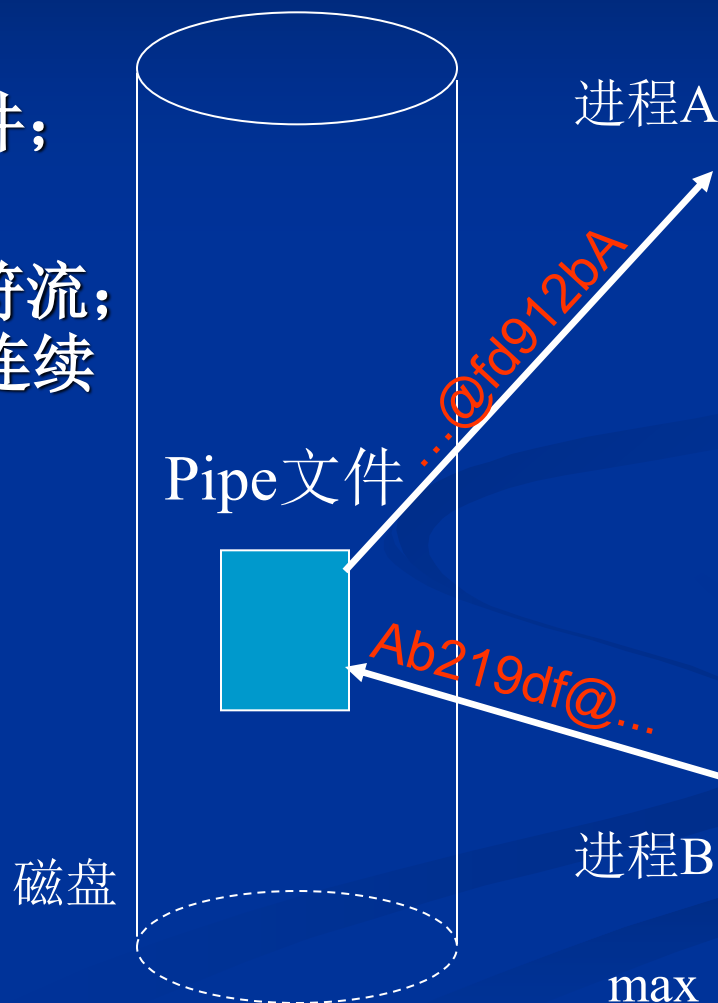
2.5.3 管道(Pipe)通信系统

■ 基本思想

在磁盘创建一个文件；
大小固定，如**4kB**；
发送进程写连续字符流；
接收进程按顺序读连续字符流；

■ 实现技术

对管道文件的共享：
同步、互斥
UNIX首创



2.5 进程通信

2.5.3 管道通信系统

■ Windows操作系统的管道通信

- Windows提供无名管道和命名管道两种管道机制。
- 利用CreatePipe可创建无名管道并得到两个读写句柄；
- 利用ReadFile和WriteFile可并行无名管道的读写。

■ CreatePipe的调用格式：

```
BOOL CreatePipe( PHANDLE hReadPipe, // 读句柄;  
                PHANDLE hWritePipe, // 写句柄;  
                LPSECURITY_ATTRIBUTES lpPipeAttributes, // 安全属性指针;  
                DWORD nSize          // 管道缓冲区字节数;  
                );
```

2.6 线程-函数级并发执行

2.6.1 线程(Thread)的基本概念

(1) 线程的引入

- 传统进程的两个基本属性

- 拥有资源的独立单位

每个进程分配一虚拟地址空间，保存进程映像，控制一些资源（文件，I/O设备），有状态、优先级

- 调度和分派的基本单位

分配CPU

- 导致的不利后果

- 身形笨重，管理和运行资源开销大。

- 限制并发度，多道程序设计道数不能太高



2.6 线程

(1) 线程的引入

■ 传统进程的两个基本属性分解

- 拥有资源的独立单位
- 调度和分派的基本单位

进程

■ 线程概念

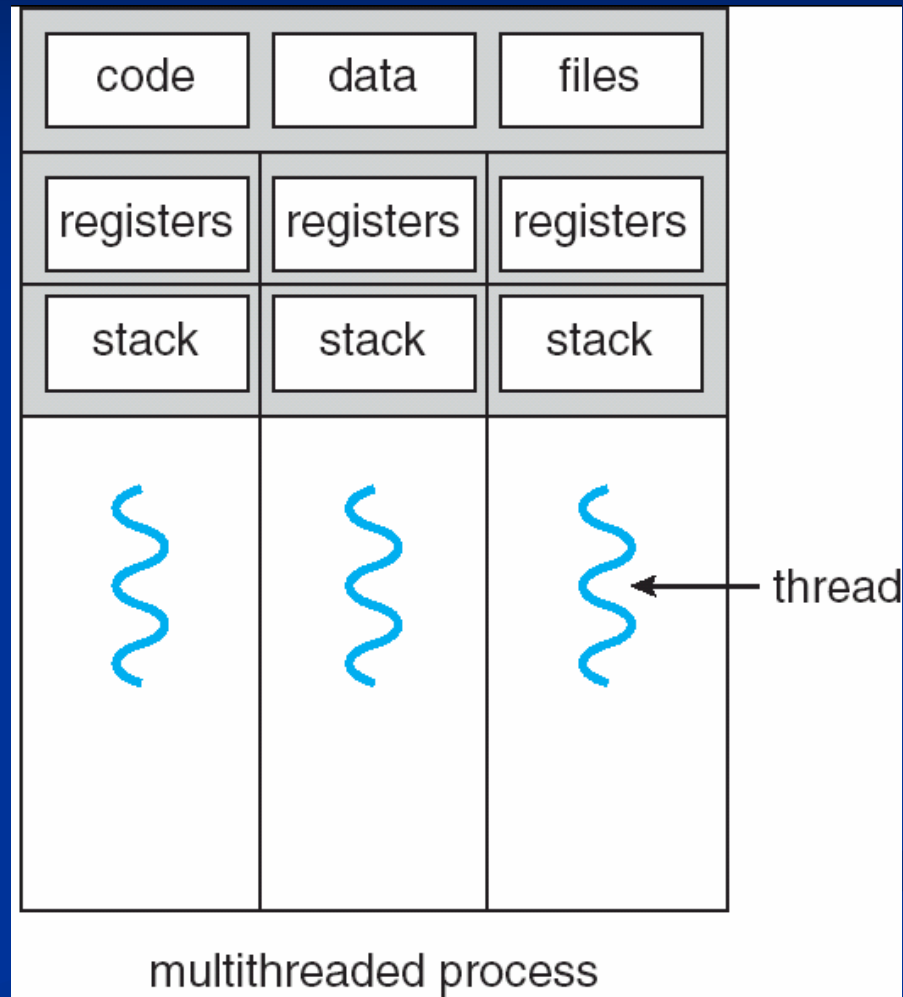
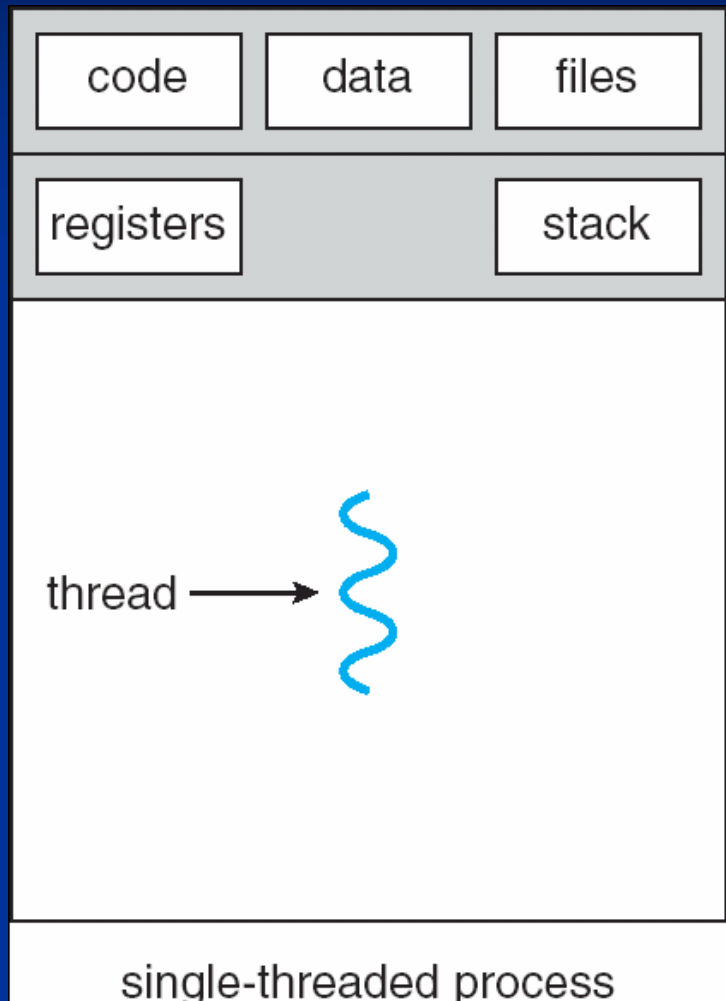
- 有时称轻量级进程
- 进程中的一个轻型运行实体
- 是一个CPU调度和分派的基本单位
- 可并发执行
- 共享进程资源

线程



2.6 线程

(2) 线程与进程的关系



2.6 线程

(3) 线程的开销

- ❑ 线程控制块 (TCB)
- ❑ 寄存器状态, 堆栈, 局部变量拷贝
- ❑ 线程运行状态 (就绪、阻塞、执行)
- ❑ 可以创建、撤消另一个线程
- ❑ 优先级与信号屏蔽

(4) 引入线程的好处

- ❑ 创建一个新线程花费时间少 (结束亦如此)
- ❑ 两个线程的切换花费时间少
- ❑ 同一进程内的线程共享内存和文件, 因此它们之间相互通信无须调用内核
- ❑ 适合多处理机系统

2.6 线程

2.6.2 线程的实现方式

- 内核支持线程

- Windows 2000/XP、Macintosh 和 OS/2 操作系统

- 用户级线程

- 一些数据库管理系统如 infomix

- 两者结合方法

- Solaris、Linux

2.6 线程

2.6.2 线程的实现方式

(1) 内核支持线程(Kernel Supported Threads, KST)

- 内核支持线程，是在内核的支持下运行，即无论是用户进程中的线程，还是系统进程中的线程，他们的创建、撤消和切换等，也是依靠内核实现。
- 在内核空间还为每一个内核支持线程设置了一个线程控制块(TCB)，内核是根据该控制块而感知某线程的存在的，并对其加以控制。
- 对于通常的进程，无论是系统进程还是用户进程，进程的创建、撤消，以及要求由系统设备完成的I/O操作，都是利用系统调用而进入内核，再由内核中的相应处理程序予以完成。进程的切换同样是在内核的支持下实现的。
- 若系统中设置的是内核支持线程，则以线程为单位进行。

2.6 线程

2.6.2 线程的实现方式

(1) 内核支持线程

- 在采用轮转法调度时，是各个线程轮流执行一个时间片。同样假定进程A中只有一个内核支持线程，而在进程B中有100个内核支持线程。此时进程B可以获得的CPU时间是进程A的100倍，且进程B可使100个系统调用并发工作。

2.6 线程

2.6.2 线程的实现方式

(2)内核支持线程

■ 优点:

- 对多处理器，核心可以同时调度同一进程的多个线程
- 阻塞是在线程一级完成
- 核心例程是多线程的

■ 缺点:

- 在同一进程内的线程切换调用内核，导致速度下降

2.6 线程

2.6.2 线程的实现方式

(2) 用户级线程(User Level Threads, ULT)

- 用户级线程仅存在于用户空间中。
- 可以为一个应用程序建立多个用户级线程。在一个系统中的用户级线程的数目，可以达到数百个至数千个。
- 用户级线程线程控制块(TCB)设置在用户空间，由用户维护，因此这种线程的创建、撤消、状态转换等功能，都无须利用系统调用来实现。而用户级线程的切换（包括同步与通信），如果发生在一个应用进程的诸多线程之间，也无须内核支持。
- 通常由专门的线程库(包)负责对线程进行管理，无须内核的帮助，因而内核完全不知道用户级线程的存在。

2.6 线程

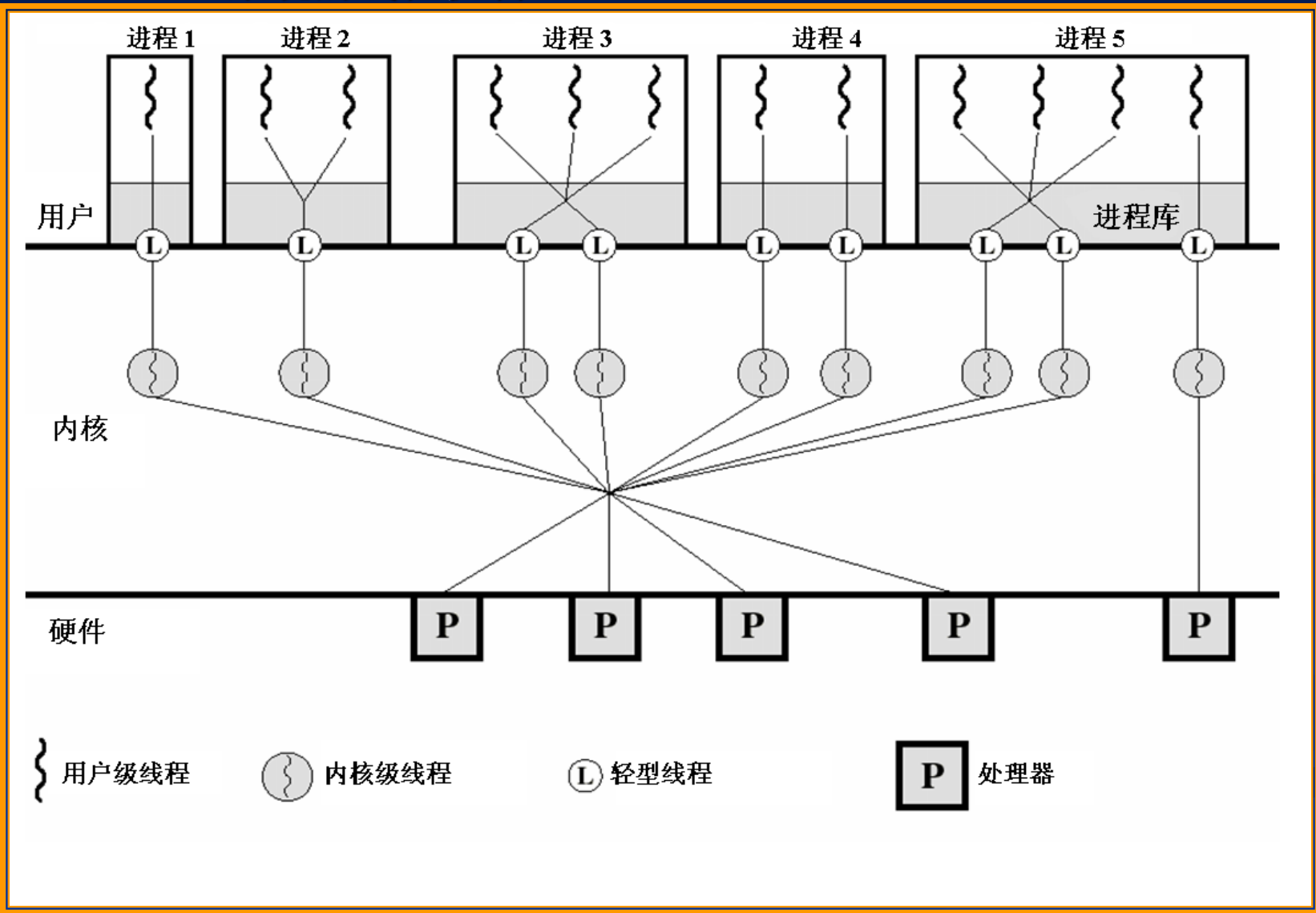
2.6.2 线程的实现方式

(2) 用户级线程

- 对于设置了用户级线程的系统，其调度仍是以进程为单位进行的。
- 在采用轮转调度算法时，各个进程轮流执行一个时间片。但假如在进程A中包含了一个用户级线程，而在另一个进程B中含有100个用户级线程，这样，进程A中线程的运行时间，将是进程B中各线程运行时间的100倍，相应地，其速度要慢上100倍。

(2) 用户级线程

■ Solaris系统的线程机制



2.6 线程

2.6.2 线程的实现方式

(2) 用户级线程

■ 优点：

- 线程切换不调用核心
- 调度是应用程序特定的：可以选择最合适(好)的算法
- ULT可运行在任何操作系统上（只需要线程库）

■ 缺点：

- 大多数系统调用是阻塞的，因此核心阻塞进程，故进程中所有线程将被阻塞
- 核心只将处理器分配给进程，同一进程中的两个线程不能同时运行于两个处理器上